

CSE 144 Final Project Report

Transfer Learning Challenge

Group members:

Tien Le (Cruz ID: 1962505)

Spring 2026

1. Introduction

For this project, I was tasked with applying transfer learning techniques in order to fine-tune a pretrained model to classify a custom dataset provided by the instructors.

With the limited time span, computing resources, and size of the provided dataset, transfer learning is a particularly appropriate approach, allowing one to take advantage of the work already done on existing models.

To this end, I utilized strategic layer-freezing, various data augmentation techniques, and some architecture changes in order to achieve a ~31.82% accuracy on the public Kaggle leaderboard.

2. Dataset

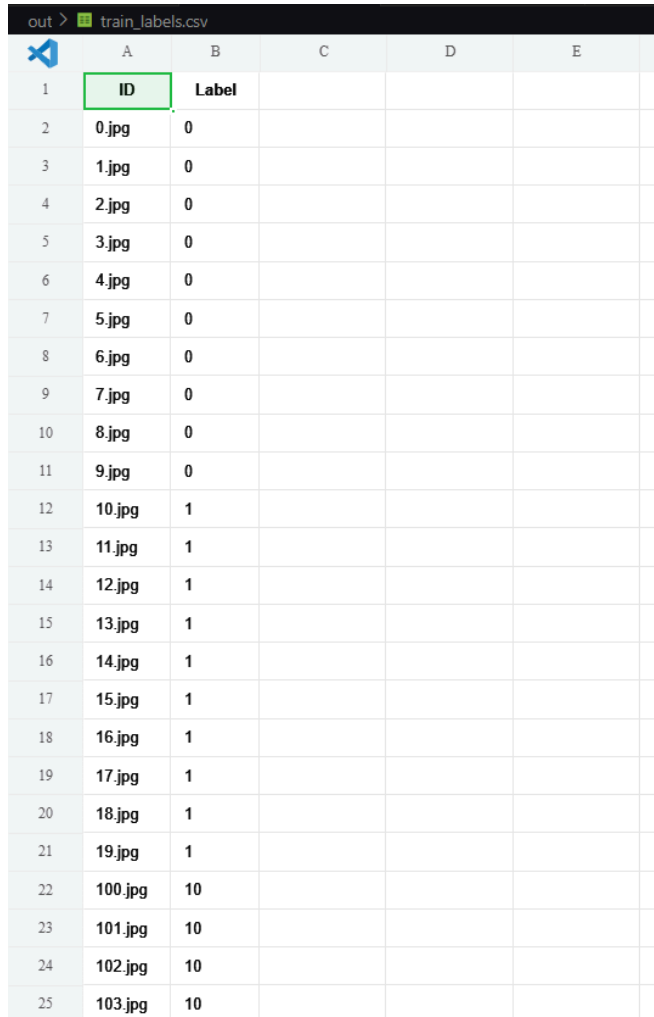
The dataset has the following features:

- Meant to represent 100 classes, spanning from specific classifications of flowers to vehicles to food
- Colored (RGB) images, of varying sizes/resolutions
- 1078 labelled training samples, meaning approximately 10 samples per label, with the number of samples varying from label to label
- 1035 unlabelled test samples

From this, I further split the labelled samples into training and validation datasets at 971 and 108 samples respectively, a roughly 9:1 split.

As provided by the instructors, the labelled training samples are provided in a directory that groups samples into folders by label, i.e. image “40.jpg” is held in folder “train/4”, meaning image 40.jpg’s intended label is 4.

To avoid scrambling labels, I wrote a .csv to act as an annotations file for the dataset, walking through the file directory to match each image file to the folder's name regardless of walk order, thereby ensuring each image has the correct label.



	A	B	C	D	E
1	ID	Label			
2	0.jpg	0			
3	1.jpg	0			
4	2.jpg	0			
5	3.jpg	0			
6	4.jpg	0			
7	5.jpg	0			
8	6.jpg	0			
9	7.jpg	0			
10	8.jpg	0			
11	9.jpg	0			
12	10.jpg	1			
13	11.jpg	1			
14	12.jpg	1			
15	13.jpg	1			
16	14.jpg	1			
17	15.jpg	1			
18	16.jpg	1			
19	17.jpg	1			
20	18.jpg	1			
21	19.jpg	1			
22	100.jpg	10			
23	101.jpg	10			
24	102.jpg	10			
25	103.jpg	10			

Above, a few rows of “train_labels.csv” showing the correctly matched labels.

Preprocessing on the training set was as follows:

- Resize to 384px*384px, with antialiasing
- RandomResizedCrop to 384px*384px, meant to break locational associations for more generalized training, with antialiasing
- RandomErasing at a rate of $p=0.1$, meant to generalize data, inspired by [Torchvision's new training recipe](#) (2021)
- Normalizing to mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]; the standard for ImageNet images

Additionally, I implemented Cutmix and Mixup after the dataloader as another data augmentation technique, with Cutmix having an alpha of 1.0 and Mixup having an alpha of 0.2. Either one or the other was applied before training at an equal probability.

3. Implementation

3.1 Model

For my pretrained backbone, I used EfficientNetV2-S, with the “IMAGENET1K_V1” pretrained weights. As the machine I am training on lacks a GPU and is not particularly strong, EfficientNetV2-S’s smaller size at 21.5M parameters would have ideally helped make the training process feasible. This, combined with the relatively strong accuracy it provided due to making use of a modified version of [Torchvision’s new training recipe](#) (2021), made it to be what I felt was a prime candidate.

I made a relatively large architecture change to this model - I swapped out the classifier head. Beyond the basic task of swapping the previous 1000 class output to 100 classes to match my problem, I added a Flatten, additional FC Layer, ReLU, and Softmax to boost the trainability of the classifier head. I experimented with different Dropout rates as well, but found the default 0.2 was the most effective.

```
(classifier): Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=1280, out_features=256, bias=True)
  (2): ReLU()
  (3): Dropout(p=0.2, inplace=False)
  (4): Linear(in_features=256, out_features=100, bias=True)
  (5): Softmax(dim=1)
)
```

Above, the new classifier head I made.

For my fine-tuning strategy, I froze all the layers except for the classifier head and trained exclusively the layers it contained for 25 epochs. I planned to unfreeze more layers in a sort of “phase 2”, but found that I had a lot of trouble with proceeding in a way that actually increased the accuracy. I additionally found that I could not unfreeze any more than the very last layer, as my CPU could not handle it.

3.2 Training

- Loss function: Cross entropy loss

- Best choice for classification task
- Optimizer: AdamW
 - Unlike Adam, weight decay is decoupled gradient calculations, making adjustments to weight decay hyperparameter more clear
- Learning rate: $5e-3$ (0.005)
 - More aggressive since the classifier head starts off untrained.
- Scheduler: Cosine Annealing LR
 - Less parameters to adjust, and [Torchvision's new training recipe](#) (2021) found that other schedulers do not provide significant improvements when Cosine Annealing is combined with checkpointing.
- Batch size: 64
- Epochs: 25
- Weight decay: $2e-5$ (0.00002)
 - Inspired by [Torchvision's new training recipe](#) (2021)
- Hardware/software environment
 - CPU: AMD Ryzen 7 5700U with Radeon Graphics
 - GPU: AMD Radeon(TM) Graphics (2 GB) - not usable by PyTorch
 - Windows 10
 - PyTorch ver. 2.8.0
 - Running locally was still faster than Google Colab in a test

4. Experiments

The baseline set-up was EfficientNetV2-S with only the singular FC layer it originally contained swapped to output 100 instead of 1000 classes. The initial training recipe was based heavily upon the specifications provided in HW2, with Adam as the optimizer and a learning rate of $1e-3$.

My hyperparameter tuning plan and validation method was to build on this recipe incrementally, adjusting based on an averaged validation loss/accuracy from running my evaluation function 5 times.

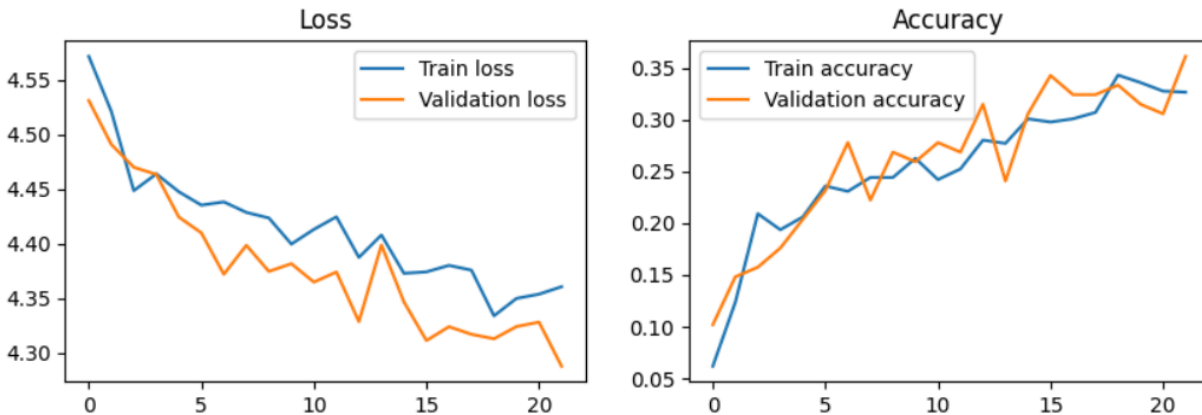
In this process, I found that...

- In the classifier-head-only training, adding more layers increased the accuracy gains dramatically.
- A higher (initial) learning rate was effective in making sure gains were being made in a reasonable amount of time, with Cosine Annealing LR scheduling making sure that this aggressive learning rate does not cause issues later in the training process.
- RandomResizedCropping and RandomErasing were introduced early in the process, with the latter demonstrating immediate strong improvement upon introduction.

- Unfreezing the lowest convolutional layers in a “phase 2” seemed to actually decrease the accuracy and even over longer epochs did not seem to provide more benefits beyond ~30%; this may be a limitation of the model or perhaps that I could not unfreeze any additional layers above that.
- While I was originally experiencing overfitting/plateauing at around or just below 30% at 15 epochs and beyond. With the introduction of Cutmix and Mixup, I found that I could get above 30% in training/validation, but at the cost of many more epochs. I was only able to test up to 25 epochs before training took well over 2 hours, but I could still have possibly gained more.

5. Results

In training, I was able to achieve a maximum training set accuracy of 34.29%¹ with a minimum loss of 4.3337; additionally, I achieved a maximum validation set accuracy of 36.11% with a minimum loss of 4.2876. These don’t necessarily correlate - the training set accuracy corresponding to the best validation stats was 32.65% corresponding to loss of 4.3604.



Above, plot showing training and validation loss/accuracy over the course of 25 epochs

Averaging validation evaluation results over 5 runs led to a best accuracy and loss of 36.11% and 4.2909, and an average accuracy and loss of 33.89% and 4.3097. Following validation evaluations of this nature were generally within 1-2 percent of these accuracy numbers, and could be accounted for by the small validation set size.

¹ Training accuracy with Cutmix and Mixup was a little difficult to calculate; ultimately, I took the approach of comparing the predicted label to the class the augmented image was apparently most like, based on the greatest value in the label tensor output. This is not a terribly precise way to measure; validation accuracy is measured as expected and theoretically is a better measure.

On Kaggle, I achieved a public submission score of 0.31818, or ~31.82%. I submitted a .csv of 1036 rows with 1 header created by looping through and evaluating the test set samples in numerical order (enforced with a lambda sort).

56	Tien Le		0.31818	2	2h
----	---------	---	---------	---	----

6. Discussion

The “why” for my techniques are fairly straightforward - more layers in the classifier head naturally leads to a greater degree of trainability, and the preprocessing/data augmentation techniques I used worked very well to help regularize the limited data set provided to maximize learning gains.

It was very interesting that attempting to train the lower layers of the model actually reduced accuracy or at the very best, did nothing. It’s possible I was unable to hit upon the recipe needed to gain significant improvements with my limitations (see below). I could not discern if by the end of 25 epochs that my model was beginning to overfit - it is totally possible that I could still get more gains by increasing epochs.

I was greatly limited by the specifications of my device - the epochs required to get results given my implemented techniques grew to over 2 hours a run, greatly slowing my iterative process. As it is my primary machine, I was also limited by the fact I could not dedicate the machine entirely to training. I also was limited in exploring model options with more parameters. I would love to be able to explore more models (I heard good results with DINO and EfficientNet_B2 in the presentations) or simply be able to increase epochs with a stronger machine.

7. Reproducibility

Random seed was set to 42, with PyTorch running on CPU. These settings and deterministic flags are the same as set on HW2 and are intended to create fixed results.

I developed this project on Python ver. 3.11.0. Package versions are as follows...

- PyTorch ver. 2.8.0
- TorchVision ver. 0.23.0
- Matplotlib ver. 3.10.0
- Scikit-learn ver. 1.6.1
- NumPy ver. 2.0.2

The first cell of the main code file, “final.ipynb”, contains a commented line to install the necessary packages if you are running this in a clean environment. If you are, make sure to uncomment the line:

```
!pip -q install torch==2.8.0 torchvision==0.23.0 matplotlib==3.10.0
scikit-learn==1.6.1 numpy==2.0.2
```

Here are step by step instructions for generating “submission.csv”:

1. Clone the repo at <https://github.com/tcle103/cse144-final>. Make sure Python $\geq 3.9.0$ is installed on your computer.
2. Open “final.ipynb”. Uncomment the above specified line if you’re running in a clean environment.
3. Run all code cells in order. Do not run the cell beginning with “# phase 1 training - train classifier head only” twice; if execution stops or you need to restart training, restart the whole kernel to reproduce results.
4. The last cell should evaluate and write “submission.csv” into a folder in the root directory called “out”.
5. That’s it!

8. Team Contributions

1. Tien Le: everything!

9. References

1. [Torchvision’s new training recipe](#) (2021)
2. [TorchVision reference for EfficientNet_V2's IMAGENET1K_V1 weights](#)
3. [TorchVision documentation on using Cutmix and Mixup](#)
4. [TorchVision documentation for general information on transforms available with the new transforms v2 API](#)
5. [Adel Bennaceur's article on Adam vs AdamW, helped me understand the differences conceptually \(did not look at or use code\)](#) (2025)